

Experiment No: 2

Title

Design distributed Application using Java RMI

Aim

To design and implement a distributed application using **Java Remote Method Invocation (RMI)** that enables a client running on one machine to invoke methods on a remote object running on another machine.

Problem Statement

Design a distributed application where a client requests an arithmetic operation (addition) from a remote server. The server performs the computation and returns the result to the client using Java RMI.

System Requirements

Hardware Requirements

- Two Virtual Machines / Two Systems
- Minimum 2 GB RAM

Software Requirements

- Java JDK 8 or above
 - Linux / Windows OS
 - Network connectivity between Client and Server
-

Theory

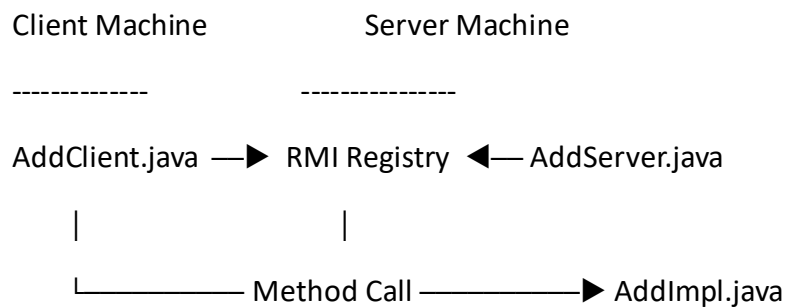
Java Remote Method Invocation (RMI) is a Java API that allows a program running in one Java Virtual Machine (JVM) to invoke methods on an object running in another JVM. RMI provides a high-level abstraction over socket programming and enables object-oriented distributed computing.

In RMI architecture:

- The **server** creates and exports a remote object.

- The **RMI Registry** maintains a mapping between object names and remote object references.
- The **client** looks up the remote object and invokes methods as if it were a local object.
- Communication between client and server is handled through **stubs and skeletons**.

Architecture Diagram



Algorithm

Server Side

1. Define a remote interface extending `Remote`.
2. Implement the remote interface.
3. Start the RMI registry on port 1099.
4. Create the remote object.
5. Bind the object to the RMI registry.

Client Side

1. Obtain the server IP address.
2. Look up the remote object using RMI registry.
3. Invoke the remote method.
4. Display the result.

Conclusion:

FAQS:

1. What is Java RMI?
2. Why does a remote interface extend Remote?
3. What is the role of RMI Registry?
4. What is a stub?
5. Why does RMI throw RemoteException?
6. Difference between RMI and Socket programming?